

Andrew Choy Special Participation A: HW8 Gemini Thinking

Conversation Log: **Sophisticated Prompt**

November 27, 2025

Overview

This document contains the step-by-step mathematical derivations for Problems 1, 3, and 4 of Homework 8, followed by a critical analysis comparing these derivations against the official solution key.

User Prompt

I'm using you to help me study for a graduate-level Deep Learning course, so I need you to act like a really rigorous TA.

Here is how I need you to answer my questions:

- **No Code:** Please don't write any Python or PyTorch. I need to understand the math, not the implementation.
- **Show Your Work:** When I give you a problem, I need a full, step-by-step mathematical derivation. Don't skip steps in the calculus or linear algebra.
- **Go Deep:** Treat this like a theory problem set. Explain the 'why' behind the derivation.

You will do problem 1, 3, and 4 from the pdf given.

Problem 1: SSM Convolution Kernel

This problem bridges the recurrent view of State-Space Models (SSMs) with the convolutional view (CNNs).

(a) Derivation of the Convolution Kernel

Goal: Express the output y_k as a convolution of the input sequence u with a kernel K .

System Dynamics:

$$\begin{aligned}x_{k+1} &= Ax_k + Bu_k \\ y_k &= Cx_k + Du_k\end{aligned}$$

Assumption: $x_0 = 0$.

Step 1: Unroll the state x_k

Let's calculate the first few terms to find the pattern:

- $k = 0$: $x_0 = 0$.
- $k = 1$: $x_1 = Ax_0 + Bu_0 = Bu_0$.

- $k = 2$: $x_2 = Ax_1 + Bu_1 = A(Bu_0) + Bu_1 = ABu_0 + Bu_1$.
- $k = 3$: $x_3 = Ax_2 + Bu_2 = A(ABu_0 + Bu_1) + Bu_2 = A^2Bu_0 + ABu_1 + Bu_2$.

Pattern identified:

$$x_k = \sum_{j=0}^{k-1} A^{k-1-j} Bu_j$$

Step 2: Substitute x_k into the output equation y_k

$$y_k = C \left(\sum_{j=0}^{k-1} A^{k-1-j} Bu_j \right) + Du_k$$

Distribute C :

$$y_k = \sum_{j=0}^{k-1} CA^{k-1-j} Bu_j + Du_k$$

Step 3: Reformulate as a convolution

A convolution is defined as $y_k = \sum_{l=0}^L K_l u_{k-l}$. We change the summation index to match this form. Let l be the “lag” or time difference: $l = k - j$.

- When $j = k$, $l = 0$. (This corresponds to the Du_k term).
- When $j = k - 1$, $l = 1$.
- When $j = 0$, $l = k$.

Substituting $j = k - l$ into the summation:

$$y_k = \underbrace{Du_k}_{\text{term where } l=0} + \sum_{l=1}^k CA^{l-1} Bu_{k-l}$$

Since inputs u with negative indices are 0, we can extend the sum to L without changing the value:

$$y_k = \sum_{l=0}^L K_l u_{k-l}$$

Step 4: Identify K

Comparing the terms, we arrive at the solution for K :

$$K_l = \begin{cases} D & \text{if } l = 0 \\ CA^{l-1}B & \text{if } l > 0 \end{cases}$$

(b) Concrete Examples

i. Scalar Case

Parameters: $n = 1, A = 0.8, B = 1, C = 1.5$. Note: $D = \delta$ is given as $D = 0$ for calculation purposes in similar contexts.

- $K_0 = D = \delta$
- $K_1 = CB = 1.5 \times 1 = 1.5$

- $K_2 = CAB = 1.5 \times (0.8) \times 1 = 1.2$
- $K_3 = CA^2B = 1.5 \times (0.8)^2 \times 1 = 0.96$
- $K_4 = CA^3B = 1.5 \times (0.8)^3 \times 1 = 0.768$

ii. 2D Case

Parameters:

$$A = \begin{pmatrix} 0.7 & 0.1 \\ 0.2 & 0.6 \end{pmatrix}, \quad B = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad C = (1 \ 0), \quad D = 0$$

Calculate Kernels ($l = 0$ to 3):

- $l = 0$: $K_0 = D = 0$.
- $l = 1$: $K_1 = CB = (1 \ 0) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 1$.
- $l = 2$: $K_2 = C(AB)$.

$$AB = \begin{pmatrix} 0.7 & 0.1 \\ 0.2 & 0.6 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.7 \\ 0.2 \end{pmatrix} \implies K_2 = (1 \ 0) \begin{pmatrix} 0.7 \\ 0.2 \end{pmatrix} = 0.7$$

- $l = 3$: $K_3 = CA(AB)$.

$$A(AB) = \begin{pmatrix} 0.7 & 0.1 \\ 0.2 & 0.6 \end{pmatrix} \begin{pmatrix} 0.7 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 0.51 \\ 0.26 \end{pmatrix} \implies K_3 = 0.51$$

Discussion: The kernel $K = \{0, 1, 0.7, 0.51, \dots\}$ represents the system's **impulse response**.

(c) Efficient Computation

Parallelization: If K is known, $y = K * u$ can be computed via FFT in $O(L \log L)$. Direct parallel summation allows $O(1)$ sequential steps (fully parallel across k) but $O(\log L)$ critical path for the summation tree. **Naive Recurrent:** Requires $O(L)$ sequential steps due to state dependency $x_k \rightarrow x_{k+1}$.

(e) & (f) Structured Matrices

- **Diagonal A:** Computation of A^k becomes element-wise power $O(n)$, making kernel generation trivial.
- **DPLR ($A = I + pp^\top$):** Eigenvalues are $(1 + \|p\|^2)$ and 1. A^k can be computed in closed form using a rank-1 update structure, avoiding $O(n^3)$ matrix multiplication.

Problem 3: Self-Supervised Linear Purification

(a) Reconstruction vs Regularization

Definitions:

- **Reconstruction Loss:** $\mathcal{L}_{rec} = \|X - WX\|_F^2$
- **Regularization Loss:** $\mathcal{L}_{reg} = \lambda \|W\|_F^2$

(b) Optimal Regularized Weight Matrix

Objective: Minimize $\mathcal{L}(W) = \|X - WX\|_F^2 + \lambda \|W\|_F^2$ given $X = U\Sigma V^\top$.

Step 1: Simplify using Unitary Invariance

The Frobenius norm is invariant under orthogonal multiplication.

$$\|X - WX\|_F^2 = \|U\Sigma V^\top - WU\Sigma V^\top\|_F^2 = \|U\Sigma - WU\Sigma\|_F^2$$

Let $\tilde{W} = U^\top W U$. Note that $\|\tilde{W}\|_F^2 = \|W\|_F^2$. The objective becomes:

$$\|\Sigma - \tilde{W}\Sigma\|_F^2 + \lambda \|\tilde{W}\|_F^2$$

Step 2: Decouple and Solve

Since Σ is diagonal, the problem separates into element-wise minimization.

- For $i \neq j$: Terms are $\tilde{w}_{ij}^2(\sigma_j^2 + \lambda)$. Minimized at $\tilde{w}_{ij} = 0$.
- For $i = j$: Let $d_i = \tilde{w}_{ii}$. Term is $(\sigma_i - d_i\sigma_i)^2 + \lambda d_i^2$.

Taking the derivative w.r.t d_i :

$$-2\sigma_i^2(1 - d_i) + 2\lambda d_i = 0 \implies d_i(\sigma_i^2 + \lambda) = \sigma_i^2 \implies d_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

Step 3: Reconstruct W

$$W = U\tilde{W}U^\top = U \begin{pmatrix} \frac{\sigma_1^2}{\sigma_1^2 + \lambda} & & \\ & \ddots & \\ & & \frac{\sigma_m^2}{\sigma_m^2 + \lambda} \end{pmatrix} U^\top$$

(c) Numerical Application

Given $\sigma = \{10, 8, 4, 1, \dots\}$. Filter function $f(\sigma) = \frac{\sigma^2}{\sigma^2 + \lambda}$.

1. **Preserve top 3** ($\sigma_3 = 4$):

$$\frac{16}{16 + \lambda} \geq 0.8 \implies 16 \geq 12.8 + 0.8\lambda \implies 3.2 \geq 0.8\lambda \implies \lambda \leq 4$$

2. **Attenuate noise** ($\sigma_4 = 1$):

$$\frac{1}{1 + \lambda} \leq 0.5 \implies 2 \leq 1 + \lambda \implies \lambda \geq 1$$

Answer: $1 \leq \lambda \leq 4$.

Problem 4: Ridge-Attention

(b) Ridge Regression Decomposition

Given $w_* = (A^\top A + \lambda I)^{-1} A^\top y$. We rewrite matrix products as sums over n samples (x_i are rows of A):

$$(A^\top A + \lambda I) = \lambda I + \sum_{i=1}^n x_i x_i^\top$$
$$A^\top y = \sum_{i=1}^n x_i y_i$$

(c) Cost of Non-Causal Ridge Self-Attention

Settings: Context n , dim d , with $d < n$.

- Compute Covariance $M = \sum k_i k_i^\top + \lambda I$: Cost $O(nd^2)$.
- Invert M^{-1} : Cost $O(d^3)$.
- Compute Cross-Covariance $R = \sum k_i v_i^\top$: Cost $O(nd^2)$.
- Solve $W_* = M^{-1} R$: Cost $O(d^3)$.
- Apply to n queries: $O(nd^2)$.

Dominant Cost: $O(nd^2)$.

(e) Sherman-Morrison Update

Formula: $(M + uv^\top)^{-1} = M^{-1} - \frac{M^{-1}uv^\top M^{-1}}{1 + v^\top M^{-1}u}$. Assuming M^{-1} is known, operations involve vector-matrix multiplications ($O(d^2)$) and outer products ($O(d^2)$). **Total Cost:** $O(d^2)$.

(f) Causal Ridge-Self-Attention

For causal attention, we must update the ridge solution at every step t . Recomputing naively is $O(n^2 d^2)$. Using Sherman-Morrison recursively:

1. **Recursive Inverse:** Update $P_t = (K_{1:t}^\top K_{1:t} + \lambda I)^{-1}$ using Sherman-Morrison with new key k_t . Cost: $O(d^2)$.
2. **Recursive Target:** Update $C_t = \sum_{i=1}^t k_i v_i^\top$ by adding $k_t v_t^\top$. Cost: $O(d^2)$.
3. **Compute Output:** $y_t = q_t^\top P_t C_t$. Cost: $O(d^2)$.

Total for sequence: $O(nd^2)$. This is linear in n , making it highly efficient compared to standard $O(n^2)$ attention.

Analysis & Corrections based on Official Solution

Problem 1(c): Critical Path Complexity

- **Initial Derivation:** Stated the critical path complexity was $O(\log L)$.
- **Correction:** The official solution specifies the complexity as $O(\log L \log n)$.
- **Reasoning:** While $\log L$ accounts for the parallel reduction of the sequence length, the additional $\log n$ factor is required to account for the computational depth of the matrix multiplication operations themselves.

Problem 1(f): DPLR Derivation Method

- **Initial Derivation:** Used an eigendecomposition approach to compute A^k .
- **Correction:** The official solution utilizes a **Binomial/Geometric series expansion**
- **Nuance:** Both methods yield correct results. However, the solution's approach ($A^l = I + p(\dots)p^\top$) is slightly more general as it does not rely on explicit assumptions about diagonalizability that the eigendecomposition method implies.

Problem 3(a): Missing Table Values

The initial derivation omitted specific numerical values for the table in 3(a). The official solution confirms the following values:

Encoder	Reconstruction Loss	Regularization Loss
α (Identity)	0	2
β	0.001	1

This confirms the conclusion that for $\lambda > 0.001$, the Identity matrix encoder ($W^{(\alpha)}$) yields a higher total loss than $W^{(\beta)}$.